PCET'S

Pimpri Chinchwad College of Engineering, Pune

Name: Soham Kulkarni **PRN:** 124B1F056 **Batch:** TY - A3

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 07

1. Problem Statement

A university must schedule course exams. If a student takes two courses, those exams cannot happen at the same time. Represent each course as a vertex and connect conflicting courses with an edge. Assign time slots, called colors, so connected courses get different slots.

2. Course Objectives

• Understand graph coloring through a simple exam timetable.

• Implement Greedy coloring and compare it with DSATUR and Welsh-Powell.

• Explain how classroom capacities add extra scheduling constraints.

3. Course Outcomes

Model common-student conflicts, assign legal slots, explain each Greedy decision, and understand the difference between a valid timetable and a guaranteed minimum-slot timetable.

4. The Basic Idea

Vertex = course. Edge = shared-student conflict. Color = exam slot. An edge means that its two courses must have different slots. Courses without an edge between them may share a slot.

The simple main program asks for course names and conflicting course pairs directly. Process the courses in ID order. For each course, mark slots already used by its scheduled neighbors and choose the first unmarked slot. This is Greedy coloring.

The full version is preserved in Extensions. It builds the graph from student enrollments, compares three algorithms, and allocates classrooms. The simple version focuses only on learning the core coloring step; its input format is different.

5. Understand the Example

Courses are 1=DAA, 2=DBMS, 3=OS, 4=CN. Enter four conflicts: (1,2), (1,3), (2,3), and (3,4). Each pair means at least one student takes both courses. All shared-student conflicts must be entered.

Course	Earlier conflicting slots	Greedy decision
DAA	None	Use slot 1
DBMS	DAA uses slot 1	Use slot 2
OS	DAA uses 1; DBMS uses 2	Use slot 3
CN	OS uses slot 3	Reuse slot 1

DAA and CN can share slot 1 because they do not conflict. DAA, DBMS and OS form a triangle: each conflicts with the other two. Therefore, at least three slots are necessary, and the example uses exactly three. This proves optimality for this example, not for every graph.

5.1. Algorithm / Pseudocode

```
Read course count and course names
Read all conflicting course pairs
Store both directions of every conflict
Set every course slot to zero (not yet scheduled)
For each course in ID order:
    Mark slots used by its scheduled conflicting courses
    Start from slot 1
    Find the first slot that is not marked
    Assign that slot to this course
Print every course and its slot
Print the total slots used
```

5.2. Main Variables

graph[i]: courses that conflict with course i. **slot[i]:** its assigned slot, with zero meaning unassigned. **used[s]:** whether a scheduled conflicting course already occupies slot s. **totalSlots:** the highest assigned slot number.

5.3. Why There Are No Clashes

When scheduling a course, all slots used by its already scheduled neighbors are blocked. The chosen slot therefore differs from every earlier neighbor. Later neighbors will perform the same check. Thus, both ends of every entered edge finish with different slots.

6. Sample Input and Output

```
Input:
4
DAA DBMS OS CN
4
1 2
1 3
2 3
3 4

Output:
Exam timetable:
ID Course Slot
1 DAA 1
2 DBMS 2
3 OS 3
4 CN 1
Slots used: 3
```

7. Performance and Algorithm Comparison

Simple Greedy: assign the first legal slot in course-ID order. This version resets a used-slot array for each course, so time is O(V squared + E), with O(V + E) memory. It accepts up to 5,000 courses. An optimized timestamp array can reduce the coloring work to O(V + E), as used in the full version.

Welsh-Powell: order courses by decreasing graph degree, then use first-fit coloring. The full implementation takes O(V log V + E) coloring time. It gives highly connected courses early priority.

DSATUR: select the uncolored course with the most distinct neighbor colors, then assign its first legal color. The full scan-based implementation takes O(V squared + E log V) coloring time and updates saturation information incrementally.

All three are heuristics: a valid coloring is guaranteed, but the minimum number of slots is not guaranteed in general. Input order can affect Greedy. The separate full program compares results and measured runtime; its timings include classroom allocation.

8. Classroom and Large-Data Extension

A conflict-free slot may contain several exams, but they also need suitable rooms. In the full extension, each exam uses one room large enough for all its students, and a room hosts at most one exam per slot. If rooms are insufficient, a color class is split into extra slots. An exam larger than every room is reported as unschedulable.

The full program reads student enrollment lists and uses adjacency sets to avoid repeated conflicts. It was tested with 3,000 courses and 6,000 students. It compares actual room-feasible slot counts rather than graph colors alone. A lower bound can certify the minimum for some inputs; otherwise, it reports a valid heuristic timetable.

9. Conclusion

The simple Greedy program assigns each course the first slot not used by a conflicting course. This gives an understandable, conflict-free timetable. The separate extension preserves DSATUR, Welsh-Powell, automatic graph construction and classroom allocation. C++ code remains separate from the report.